

PROYECTO FINAL M1 — WEB FOUNDATIONS & AGENTIC CODING

TechFix Agenda

Documentacion tecnica del sitio: estructura de archivos,
contenido de cada pagina y las decisiones de
diseno/desarrollo detras de ellas.

Repositorio: github.com/jcartagenadevit/techfix-agenda-espol

1. Arquitectura general del sitio

El proyecto es un sitio estatico multipagina, sin framework ni backend: tres archivos HTML (`index.html` , `industrias.html` , `certificaciones.html`) comparten una unica hoja de estilos (`style.css`) y el mismo encabezado/pie de pagina. Un unico script (`main.js` , compilado desde `main.ts`) valida el formulario de tickets.

Por que multipagina en vez de una sola pagina larga: cada pagina obtiene su propio `<title>` y sus propias etiquetas `meta description` / Open Graph, lo cual mejora el SEO tecnico (cada URL es indexable por separado) y evita que el usuario tenga que hacer scroll por contenido que no le interesa (por ejemplo, alguien que solo quiere ver las certificaciones del equipo).

Por que sin framework ni build tool (salvo TypeScript): el alcance del proyecto no requiere estado complejo ni ruteo dinamico. Un sitio estatico se despliega directo en GitHub Pages sin pipeline de build, reduce puntos de falla y es mas facil de auditar para fines academicos.

2. index.html

Es la pagina principal ("landing"). Contiene tres secciones ancladas: `#inicio` (hero), `#servicios` (catalogo de 3 servicios) y `#agendar` (formulario de apertura de tickets).

Estructura semantica y accesibilidad

Usa elementos HTML5 con roles ARIA explicitos para maximizar compatibilidad con lectores de pantalla:

```
<header role="banner">
<nav aria-label="Navegacion principal">
<main id="main-content" role="main">
<footer role="contentinfo">
```

Skip link + `.sr-only` : el enlace `Saltar al contenido principal` es invisible hasta recibir foco de teclado. Permite a usuarios de teclado/lectores de pantalla evitar repetir la navegacion en cada pagina, un requisito directo de la rubrica de accesibilidad de la Tarea 2.

Formulario con validacion accesible

El formulario usa `novalidate` para desactivar la validacion nativa del navegador y delegar toda la logica a TypeScript (`main.ts` → `main.js`), lo que permite controlar el mensaje de error, el estilo y el momento exacto en que aparece:

```
<input type="email" id="email" required
      aria-required="true" aria-invalid="false"
      aria-describedby="email-error">
<span id="email-error" class="error-message" aria-live="polite"></span>
```

Por que `aria-live="polite"` : cuando TypeScript inserta el texto de error, un lector de pantalla lo anuncia automaticamente sin necesidad de que el usuario navegue hasta ese elemento. `aria-invalid` se alterna entre "true"/"false" para que el estado del campo sea perceptible tanto visualmente (CSS lo pinta de rojo) como programaticamente.

Categorías del `<select>` alineadas con el resto del sitio: las 4 opciones (Hardware, Sistemas Operativos, Redes y VPN, Gestion de Dispositivos/MDM) son exactamente las mismas 4 categorías que se usan en `certificaciones.html` . Esto mantiene un vocabulario de servicios consistente en todo el sitio en vez de inventar nombres distintos en cada página.

3. `industrias.html`

Página secundaria que responde a "a quien atendemos": muestra 4 industrias (Salud y Clínicas, Educación, Finanzas y Banca, Retail y Comercio) en tarjetas dentro de una grilla (`.grid-industries`).

Grid de 1 -> 2 columnas (no 4) en escritorio: con solo 4 tarjetas, forzar 4 columnas dejaría el texto de cada descripción muy comprimido en una sola línea. Dos columnas de dos filas mantiene párrafos legibles sin sacrificar el barrido visual rápido.

Enlaces del nav hacia `index.html#ancla` : como las secciones `#inicio` , `#servicios` y `#agendar` solo existen físicamente en `index.html` , los enlaces del menú en esta página apuntan de vuelta a esa página con el ancla correspondiente, en vez de un simple `#ancla` que fallaría silenciosamente.

Reutiliza íntegramente el mismo `header` / `footer` /skip-link que `index.html` , apoyándose en el mismo `style.css` : cero CSS duplicado entre páginas.

4. [certificaciones.html](#)

Muestra 8 perfiles de tecnicos, dos por cada una de las 4 categorias de servicio del formulario, cada uno con nombre, especialidad, una breve descripcion y sus certificaciones oficiales como badges.

Categoria (igual que el select)	Tecnicos	Certificaciones
Soporte de Hardware	Carlos Andrade, Mariana Solis	CompTIA A+, CompTIA Server+, HP ASE
Sistemas Operativos	Diego Herrera, Valentina Rojas	MS Windows Server, Apple ACSP, LPIC-1
Redes y VPN	Andres Pena, Camila Torres	Cisco CCNA, CCNA Security, CompTIA Network+/Security+
Gestion de Dispositivos (MDM)	Javier Munoz, Paola Vintimilla	MS Endpoint Admin Associate, Apple Deployment (Mosyle)

Por que 8 perfiles y no una lista generica de certificaciones: conecta narrativamente el formulario de tickets con personas reales detras de cada categoria — cuando alguien elige "Redes y VPN" en el select, esta pagina le muestra que existe un especialista certificado (Cisco/CompTIA) responsable de ese tipo de ticket. Refuerza credibilidad sin depender de precios.

Avatares con iniciales generados en CSS (no fotos): un `<div class="avatar">` con fondo en gradiente y las iniciales del nombre evita depender de imagenes externas (stock photos, licencias, peso de archivos) y mantiene el sitio 100% autocontenido, igual que el resto del proyecto (sin llamadas a APIs de imagenes).

Certificaciones como lista semantica (`<ul aria-label="Certificaciones de {nombre}">`): un lector de pantalla anuncia "lista de N elementos, certificaciones de Carlos Andrade", dando contexto explicito en vez de una simple secuencia de `<span>` sueltos.

5. Meta.md

Documenta, fuera del HTML, el proposito de cada etiqueta `<meta>` usada para SEO tecnico y para compartir en redes sociales (Open Graph y Twitter Card), requisito explicito de la Tarea 2.

Etiqueta	Proposito
<code>charset</code>	Codificacion de caracteres UTF-8 (indispensable para tildes y enies)
<code>viewport</code>	Habilita disenio responsive en moviles
<code>description</code>	Texto que Google muestra debajo del titulo en resultados de busqueda
<code>og:title</code> / <code>og:description</code> / <code>og:image</code>	Controlan como se ve la tarjeta de previsualizacion al compartir el link en WhatsApp, Facebook, LinkedIn, etc.
<code>twitter:card</code>	Formato de la tarjeta de previsualizacion especifico de X/Twitter

Por que un archivo `.md` aparte y no solo comentarios en el HTML: sirve como referencia rapida para quien audite el proyecto (por ejemplo, al calificar la tarea) sin tener que leer las >100 lineas de cada HTML para verificar que las etiquetas SEO existen y estan completas.

6. robots.txt

```
User-agent: *  
Allow: /  
Disallow: /admin/  
Disallow: /private/  
  
Sitemap: https://tusitio.com/sitemap.xml
```

User-agent: * / **Allow: /** : autoriza a todos los motores de búsqueda a indexar la totalidad del sitio público.

Disallow: /admin/ y **/private/** : rutas reservadas de forma preventiva. Aunque hoy el sitio no tiene panel de administración, dejar esta convención documentada evita tener que recordarla si en el futuro se agrega una, y es una práctica estándar recomendada aun en sitios simples.

Sitemap: : apunta a un **sitemap.xml** bajo un dominio placeholder (**tusitio.com**), pendiente de reemplazar por el dominio real una vez el sitio tenga uno propio (por ejemplo, el subdominio de GitHub Pages).

7. style.css

Unica hoja de estilos compartida por las tres paginas. Se organiza en: tokens de disenio, reset/base, accesibilidad, layout (header/nav/main), componentes (tarjetas, formulario, botones) y media queries.

Design tokens

```
:root {
  --primary-color: #0f172a;
  --secondary-color: #3b82f6;
  --gradient-brand: linear-gradient(135deg, #3b82f6, #8b5cf6, #ec4899);
  --radius: 8px;          /* controles de formulario */
  --radius-lg: 20px;      /* tarjetas / hero */
  --radius-pill: 999px;   /* botones / nav / badges */
  --spacing-sm/md/lg ...
}
```

Por que variables CSS centralizadas: requisito explicito de la Tarea 2 ("CSS con variables/design tokens"). Practicamente, cambiar el color de marca o el radio de borde en un solo lugar lo propaga a las tres paginas sin tocar el HTML.

Dos escalas de `border-radius` (`--radius` vs `--radius-lg` / `--radius-pill`): los controles de formulario (input, select, textarea) mantienen un radio pequeno y "funcional", reconocible como elemento de entrada de datos; las tarjetas, botones y el nav usan radios grandes o tipo pildora, reservando el look mas atrevido del redisenio para elementos de marketing/marca.

Mobile-first con un unico breakpoint funcional

Todos los estilos base estan pensados para pantallas angostas; un solo `@media (min-width: 768px)` reorganiza el nav de columna a fila y cambia las grillas de 1 a 2/3/4 columnas segun la pagina.

Flexbox vs Grid, cada uno donde corresponde: Flexbox se usa para layouts de una dimension (la barra de navegacion, el formulario en columna); CSS Grid se usa para colecciones de tarjetas (servicios, industrias, equipo), donde se necesita alinear filas y columnas a la vez. Es la herramienta correcta segun el tipo de layout, no una eleccion arbitraria.

Accesibilidad en CSS

```
.skip-link { position: absolute; top: -100%; }
.skip-link:focus { top: 0; }

.sr-only {
  position: absolute; width: 1px; height: 1px;
  overflow: hidden; clip: rect(0,0,0,0);
}

input[aria-invalid="true"] { border-color: var(--accent-color); }
```

.sr-only vs display:none : **.sr-only** saca el texto de la vista sin quitarlo del arbol de accesibilidad, para que un lector de pantalla si lo lea (por ejemplo, las instrucciones del formulario). **display:none** lo hubiera ocultado tambien para el lector de pantalla, que es exactamente lo que no se quiere.

Estilos ligados a atributos ARIA ([aria-invalid="true"] , [aria-current="page"]): el estado visual (rojo en campo invalido, resaltado en gradiente en el link activo del nav) esta atado al mismo atributo que ya usan los lectores de pantalla, evitando mantener dos fuentes de verdad (una clase CSS y un atributo ARIA por separado que podrian desincronizarse).

Rediseño visual (inspirado en Revolut)

Nav flotante tipo "pildora" (position: sticky + border-radius-pill): imita el patron de barra de navegacion flotante comun en sitios fintech modernos, manteniendo el menu siempre visible sin ocupar todo el ancho de la pantalla.

Hero oscuro con gradientes radiales ("blobs"): `background-image` con dos `radial-gradient` semitransparentes sobre `--primary-color` crea profundidad visual sin necesidad de imagenes ni JavaScript, y sigue siendo una simple regla CSS, liviana de cargar.

Tarjetas con borde superior en gradiente y elevacion al `:hover` : `border-image: var(--gradient-brand) 1` reemplaza el borde solido original; `transform: translateY(-4px)` en hover da retroalimentacion visual de interactividad sin JavaScript adicional.

Contraste de texto en el hero (`--text-inverted` en vez de reutilizar `--text-light`): `--text-light` (#94a3b8) esta calibrado para contraste sobre fondo blanco; sobre el fondo oscuro del hero no cumple un contraste comodo, por eso se agrego un token separado (#e2e8f0) pensado especificamente para texto secundario sobre fondo oscuro.

Resumen de decisiones tecnicas transversales

- **Sitio estatico multipagina sin build tool** (salvo TypeScript → JavaScript): apropiado para el alcance academico, se despliega directo en GitHub Pages.
- **Una unica hoja de estilos** compartida por las tres paginas: evita duplicacion y garantiza consistencia visual.
- **TypeScript solo donde hay logica real** (validacion del formulario): tipado de elementos del DOM, se compila una vez a `main.js`.
- **Vocabulario de servicios consistente** entre el `<select>` del formulario, la pagina de industrias y la de certificaciones.
- **Accesibilidad como eje transversal**, no solo en el formulario: roles ARIA, `aria-current`, `aria-label` descriptivos y contraste cuidado en el hero oscuro.
- **Sin dependencias externas de imagenes:** avatares e iconografia resueltos con CSS puro (gradientes, iniciales), manteniendo el sitio autocontenido y liviano.